Question

Write a Java program to demonstrate interface inheritance.
Create two interfaces I2 and I3 containing add() and sub() methods. Extend both interfaces using interface I1, add a mul() method, and implement all the methods in class C1.

Steps for Execution

1. Create interface I2 with the method add().
2. Create interface I3 with the method sub().
3. Create interface I1 that extends both I2 and I3.
4. Declare the mul() method inside I1.
5. Create class C1 that implements interface I1.
6. Declare a = 10 and b = 5.
7. Implement add() to calculate addition.
8. Implement sub() to calculate subtraction.
9. Implement mul() to calculate multiplication.
10. Create an object c of class C1.
11. Call add(), sub(), and mul().
12. Display the results.

Algorithm

1. **Start**
2. Define interface I2 with method add().
3. Define interface I3 with method sub().
4. Define interface I1 extending I2 and I3.
5. Add method mul() to I1.
6. Define class C1 implementing I1.
7. Initialize a = 10 and b = 5.
8. Implement add(), sub(), and mul().
9. Create an object of C1.
10. Call all three methods.
11. Display the results.

12. **Stop**

UML Class Diagram

```
┌───────────────────┐
|      I2         |
|  <<interface>>   |
├───────────────────┤
| + add() : void    |
└─────────┬─────────┘
        |
        |
        ▼
┌───────────────────┐
|      I1         |
|  <<interface>>   |
├───────────────────┤
| + mul() : void    |
| + add() : void    |
| + sub() : void    |
└─────────▲─────────┘
        |
        | implements
        |
┌─────────┴─────────┐
|      C1         |
├───────────────────┤
| a : int = 10      |
| b : int = 5       |
├───────────────────┤
```

```
| + add() : void    |
| + sub() : void    |
| + mul() : void    |
| + main() : void   |
└─────────────────┘
        ▲
        |
┌─────────────────┐
|      I3         |
|   <<interface>>   |
├─────────────────┤
| + sub() : void    |
└─────────────────┘
```

A simpler way to understand the relationship is:

```
      I2          I3
       \         /
        \       /
         └──> I1 <──┘
              |
              | implements
              ↓
             C1
```

Here, I1 **inherits from both** I2 **and** I3, while C1 implements I1. This is **multiple inheritance through interfaces**.

Code

Java Interface Inheritance

package javapractice;

```java
interface I2 {
    void add();
}

interface I3 {
    void sub();
}

interface I1 extends I2, I3 {
    void mul();
}

class C1 implements I1 {
    int a = 10, b = 5;

    @Override
    public void add() {
        System.out.println("Addition = " + (a + b));
    }

    @Override
    public void sub() {
        System.out.println("Subtraction = " + (a - b));
    }

    @Override
    public void mul() {
        System.out.println("Multiplication = " + (a * b));
```

```java
    }

    public static void main(String[] args) {
        C1 c = new C1();

        c.add();
        c.sub();
        c.mul();
    }
}
```

Output

Addition = 15

Subtraction = 5

Multiplication = 50